

从第一性原理思考 Agentic Engineering

原创 魏依承 腾讯云开发者 2026年4月23日 08:46 北京

关注腾讯云开发者，一手技术干货提前解锁🔑



腾讯云开发者

腾讯云官方社区公众号，汇聚技术开发者群体，分享技术干货，打造技术影响力交...
1095篇原创内容

公众号

本文中的方法论已经落地为一个有真实项目使用经验的开源框架，欢迎大家 fork/contribute：

agentic-engineering-framework (<https://github.com/davidYichengWei/agentic-engineering-framework>) — 基于 Skill 的模块化 Agentic Engineering 框架，包含完整的 SDLC Workflow、Best Practices、Self-Refinement 机制及项目定制指南。

AI 正在深刻改变软件开发的方式。从最初的代码补全，到如今的自主式 AI Agent，开发者与 AI 的协作模式正在快速演进。在这个过程中，一种被称为 **vibe coding** 的实践模式率先流行——开发者将需求直接抛给 AI，不审查 diff、不理解生成的代码，凭直觉接受输出，以最快的速度得到“能跑”的结果。Vibe coding 在原型验证和个人项目中有其价值，但它的本质是用速度换取了理解和控制，无法承载生产级系统的质量要求。

Agentic Engineering 代表了一种截然不同的范式 [1]。它是一种工程师与 AI Agent 深度协作的模式——AI 不仅是代码的执行者，也是问题分析、方案设计等环节的思考伙伴；但**最终的判断和决策权始终在工程师手中**。它不是“让 AI 替你写代码”，而是**将工程纪律与 AI 能力系统性地结合**，在保持甚至提升质量标准的前提下，大幅提升研发效能。

这里要强调 **Engineering（工程）** 一词的含义：Engineering 的本质是**约束优化**——在给定的资源、时间、质量等约束下，找到最优可行解。Agentic Engineering 就是在这一完整框架下引入 AI Agent 作为协作者，保留约束、保留质量标准，来提升工程过程的效率和能力。

本文采用第一性原理的思路来思考 Agentic Engineering：不从业界实践归纳“怎么做”，而是从基本事实出发，演绎推导“为什么这样做，以及怎样做才是最优的”。



为什么用第一性原理？

在进入具体讨论之前，先说清楚我们用什么方法思考，以及为什么选择这种方法。

第一性原理思维（First Principles Thinking）是一种将问题拆解到最基本的、不可再分的事实，然后从那里重新构建推理的思考方法——不是“别人怎么做，我就怎么做”（归纳），而是回到根本去追问“什么是对的，什么是对的”（演绎）。具体而言：先**解构问题**至不可争辩的基本事实，再**识别并质疑**现有方案背后的隐含假设，最后仅基于基本事实**从零重建**解决方案。

本文的立场是：归纳与演绎互补，不是替代。 归纳法（观察案例 → 总结规律）告诉我们“别人怎么做”，演绎法（从公理 → 推导结论）告诉我们“为什么这样做，以及能否做得更好”。本文以演绎法为主线，从基本事实推导 Agentic Engineering 的最佳实践，同时在关键节点引用业界实践作为归纳验证。



问题与机遇

软件工程的固有挑战是什么？AI 带来了哪些机遇？又引入了什么新问题？

1.1 软件工程的固有挑战

软件开发生命周期（SDLC）的每个阶段都存在顽固的痛点——它们不是某个团队的特殊困境，而是软件工程长期以来的共性问题：

需求阶段：意图表达失真。 需求文档质量参差不齐，往往描述“怎么做”而非“要解决什么问题”——缺少对目标、约束和成功指标的清晰定义。需求变更频繁且缺乏追踪，导致文档与实际交付物脱节。

设计阶段：知识难以沉淀与复用。 设计文档缺失或散落在各处 Wiki 中，与代码快速脱节。缺乏可复用的设计原则清单，方案的完备度高度依赖个人经验——性能、可用性、可扩展性等维度的考量因人而异。

编码阶段：上下文获取成本高。 大型项目（尤其是 C++ 项目）的上下文极其复杂，接手陌生模块或维护遗留代码的理解成本很高。团队特有的编码规范和最佳实践（如特定的锁机制、内核修改注释约定）靠人工记忆，执行依赖 Code Review，容易遗漏。

测试阶段：编写成本高，质量难保证。 编写单元测试繁琐——需要 Mock 复杂依赖、构造测试数据，分布式系统尤甚。边界条件和异常场景的覆盖率长期不足。

审查与排查阶段：瓶颈在人。 Code Review 耗时长，不熟悉模块的 reviewer 往往只能关注表面问题，架构和逻辑缺陷难以发现。现网问题排查依赖值班人员对特定模块的熟悉度，排查经验未被系统化沉淀。

这些问题的共同根源可以归结为四点：

- **信息损耗：**意图在从构想到代码的过程中逐步失真——需求被误解、设计有歧义、代码偏离原意。（主要体现于需求和设计阶段）
- **知识孤岛：**经验存在于个人头脑中，难以系统化传承和复用——设计决策的上下文、模块的隐含约束、排查经验都随人员流动而流失。（贯穿设计、编码和审查排查阶段）
- **认知成本：**理解复杂系统需要大量认知投入——即使文档齐全、知识充分，审查他人代码、理解陌生模块仍然消耗大量注意力和判断力。（主要体现于编码和审查阶段）
- **重复性劳动：**编写测试、构造数据、Mock 依赖等工程活动需要大量机械性但不可省略的工作，长期占用工程师的时间和精力。（主要体现于编码和测试阶段）

它们不是管理问题，而是软件工程的结构性困境——几十年来一直存在，只是被不同程度地缓解，从未被根本解决。

1.2 AI 的双面效应：改善与新挑战并存

AI 对上述四个维度的影响并非单向的——在每个维度上，AI 同时带来了改善和新的结构性挑战：

信息损耗：降低旧损耗，引入新损耗。 AI 缩短了从想法到可运行代码的反馈周期，降低了传统的信息损耗率。但 AI 的输出是概率性的——同样的输入可能产生不同的输出，生成的代码可能表面正确但语义偏差。这引入了一种**新类型**的损耗：不是人类的误解，而是 AI 的“似是而非”。

知识孤岛：通用知识丰富，私有知识仍然缺失。 AI 在训练数据中积累了广泛的通用知识（语言、框架、算法、设计模式），可以即时提供跨领域的知识参考。但团队的隐性知识——编码规范的微妙之处、模块的历史决策原因、Bug 排查技巧——AI 一无所知。**AI 只能利用它能“看到”的知识。** 如果团队经验没有被结构化地沉淀为 AI 可访问的形式，AI 就只是一个“什么都不知道的聪明实习生”。

认知成本：释放与负担同时发生。 AI 释放了工程师在“工具层面”消耗的认知带宽（记忆语法、查阅 API、编写样板代码）。然而，AI 同时也大幅增加了需要判断和审查的信息量——当 AI 每小时生成数百行代码等待审查时，问题不再是“写不出来”，而是“看不完、想不清”。**净效果取决于我们如何管理这个平衡。**

重复性劳动：生成成本骤降，验证成为新瓶颈。 这是 AI 改善最直接的维度——代码生成、测试生成、Mock 构造等机械性工作可以被大幅自动化。业界数据已验证了这一点：GitHub 的研究表明开发者编码速度提升了 55% [2]；Anthropic 的内部调研显示员工自报生产力提升 50%，且约 27% 的 AI 辅助工作是“原本不会做的任务”[3]。然而，验证代码正确性的成本并未同步降低——你仍然需要理解代码在做什么、判断它是否符合设计意图、审查边界条件。**生成能力的爆炸式增长与验证能力的相对停滞之间的张力，是 AI 时代软件工程的核心矛盾。**

1.3 小结

维度	固有挑战	AI 的改善	AI 引入的新问题
信息损耗	意图逐步失真	缩短反馈周期	概率性输出引入“似是而非”损耗
知识孤岛	隐性知识难以传承	通用知识即时可用	团队私有知识仍无法被 AI 利用
认知成本	理解复杂系统消耗大量认知	辅助代码解读	审查信息量暴增，释放与负担并存
重复性劳动	机械性工作占用大量时间	大幅自动化	生成成本骤降但验证成本未降

这些挑战互相强化：验证瓶颈加剧认知负担，认知过载导致工程师没有精力沉淀知识，知识孤岛又使得 AI 无法有效辅助验证。要打破这个循环，我们需要一个系统性的工程化策略——这正是后续章节要推导的内容。

02
目标

定义“最优策略”的含义——我们究竟在优化什么？

回到导语中提到的核心定义：**Engineering 是在约束条件下寻找最优可行解的系统性过程。**这意味着，当我们谈论“目标”时，不是在问“AI 能做什么”，而是在问“在给定的质量、安全、可维护性等约束下，如何利用 AI 最大化团队的总产出价值”。这正是工程化思维与 vibe coding 的根本区别——后者只追求速度这一个维度，而 Engineering 要求我们同时考虑整个约束空间。

2.1 三层价值模型

AI 对软件工程的價值可以分为三个递进的层次：

层次	名称	含义	典型场景
L1	加速 (Accelerate)	同样的事做得更快	写脚本、生成样板代码、格式转换、简单 CRUD
L2	增强 (Augment)	同样的事做得更好	提升代码质量、更全面的测试覆盖、更严谨的设计评审
L3	解锁 (Unlock)	做以前做不到的事	系统性知识沉淀与复用、跨模块的架构级分析、新工程师快速达到团队水准

2.2 本文的聚焦：L2 与 L3

一个关键观察是：**L1 级别的价值，现有的 AI 模型已经可以在不需要太多体系化方法的情况下很好地实现。** 写一个脚本、生成一段样板代码、做格式转换——这些任务的共同特点是：约束少、上下文简单、正确性容易验证。你只需要打开 AI 助手，描述需求，检查输出即可。Vibe coding 在这个层面是完全够用的。

真正需要 Engineering 思维的，是 L2 和 L3 级别的任务。这些任务的共同特点是：

- **约束空间复杂：**不仅要“能跑”，还要满足性能、安全、可维护性、团队规范等多维约束
- **上下文庞大：**涉及大型代码库的跨模块交互、历史决策原因、隐性团队知识
- **验证困难：**正确性不是运行一下就能确认的，需要深入的设计审查和全面的测试

这正是导语中 Engineering 定义的核心要义：**当约束空间复杂到无法凭直觉应对时，我们需要系统性的方法论。**

因此，本文的目标可以精确表述为：

推导并构建一套系统性的方法论，使 AI Agent 能够在复杂约束条件下可靠地提升软件工程质量，并拓展工程师的能力边界，而非仅仅加速现有流程。

2.3 如何衡量“最优”

明确了聚焦层次后，我们需要定义“最优策略”的衡量标准。具体而言，“最优”意味着在以下维度上取得最佳权衡：

- **效率与质量同步提升：**不是用质量换速度，而是通过系统性的方法让两者同时改善——AI 承担繁重的执行工作，工程师将释放出的认知带宽投入到更严谨的设计和审查中
- **解放并重新分配认知带宽：**将工程师从繁杂的基础性工作（查阅 API、编写样板代码、构造测试数据）中解放出来，使其能够专注于问题的本质——定义问题、分析权衡、做出判断；同时确保

AI 生成量的增长不超过工程师的审查和验证能力，避免认知过载

- **知识可沉淀：**工程过程中产生的知识和决策应当被系统化地保存，而非随对话消散
- **可持续演进：**方法论本身应当能够随着 AI 能力的提升和团队经验的积累而持续改进

03

第一性原理推导

从第一性原理出发：先识别不可争辩的基本事实作为公理，再以这些公理为依据批判性地审视业界假设，最终推导出核心最佳实践。

3.1 基本原理（公理）

以下三条公理是本文后续所有推导的地基。它们分别定义了人机协作的三个基本要素——**活动**（我们在做什么）、**工具**（我们用什么）、**人**（谁在做）——各自独立，无法互相推导。如果你不同意其中任何一条，后续的结论也将不成立。

公理 1：软件工程中的信息损耗（Information Loss in Software Engineering）

软件工程的本质是将人类头脑中模糊的意图，通过一系列信息变换，逐步精确化为机器可执行的代码。这条链的每一步都可能引入信息损耗。

第 1 章列举的 SDLC 各阶段痛点，表面上看是各自独立的问题。但如果我们追问它们的共同本质，会发现一个统一的模型——我们称之为**意图转化链**：

人类意图 → 自然语言需求 → 结构化设计 → 形式化代码 → 可执行程序

软件工程的本质，就是沿着这条链将模糊的意图逐步精确化。链的每一步都是一次信息变换，每一次变换都可能引入信息损耗——需求阶段的"意图表达失真"、设计阶段的"知识难以沉淀"、编码阶段的"上下文获取成本高"，都是这条链上不同位置的损耗表现。

无论工具如何演进——从打孔卡到汇编到高级语言到 AI——这条链始终存在，只是各环节的成本和损耗率在变化。AI 的引入并没有消除这条链，而是改变了链上各环节的执行方式和损耗率。几十年来软件工程积累的方法论（需求评审、设计文档、代码审查、测试体系），本质上都是在这条链的各个环节设置**校验点**，捕获和修复损耗。这一点在 AI 时代不仅依然成立，而且更加重要——因为 AI 的概率性本质引入了新的损耗来源。

公理 2：LLM 的本质特征 (The Nature of LLM)

LLM 是一个基于上下文进行概率性推理的系统，具有三个并列的本质特征：(1) 输出由上下文决定，(2) 输出是概率性的，(3) 工作记忆是有限且易失的。

AI Agent 的核心引擎是 LLM——工具调用、工作流编排等都是围绕 LLM 构建的工程层。Agent 的能力上限和本质局限，最终都由 LLM 决定。因此，这条公理聚焦于 LLM 本身的特征。LLM 通过海量训练数据获得了广泛的通用知识，能够理解和遵循复杂指令，并进行跨领域的推理和知识迁移——这构成了它作为协作者的能力基础。但与此同时，三个本质特征各自独立，共同构成了我们使用 LLM 时必须遵循的约束：

- **上下文决定性**：LLM 的输出严格由其接收到的上下文决定——给它什么信息，它就基于什么推理。它对训练数据之外的知识（团队业务逻辑、内部架构决策、编码规范）掌握为零——不是“不够聪明”，而是“不知道”。对于工程团队而言，模型能力通常是给定常量，真正的提效杠杆在于上下文的质量和相关性。
- **概率性**：LLM 本质上是基于概率的“预测下一个 Token”。同样的输入可能产生不同的输出，这不是缺陷而是架构层面的本质属性。其推理能力从训练数据中涌现而非被显式编程，因此能力边界难以精确预测，输出不可保证正确。
- **工作记忆的有限性与易失性**：LLM 的上下文窗口既有容量上限，又是易失的 (Ephemeral)——会话结束后，讨论结论和决策历史直接丢失，没有跨会话的持久记忆。会话内同样存在挑战：当上下文超出窗口限制时，Agent 框架通常采用 compaction 机制压缩信息，但哪些信息“关键”本身就是一个有损判断。即使上下文没有被截断，超长上下文也会导致“Lost in the Middle”现象——模型对中间位置信息的回忆准确度显著下降 [4]。

这三个特征的**组合效应**——而非单个特征——产生了一系列对工程实践有深远影响的结论（如不确定性的来源、错误的累积效应、上下文持久化的必要性等），将在后续章节中正式推导。

公理 3：人类认知是稀缺资源 (Human Cognition as Scarce Resource)

工程师的注意力、判断力和决策能力是有限的，是整个人机协作系统的瓶颈。

这条公理定义了人的本质约束。这是认知心理学的基本结论（工作记忆容量有限、注意力是零和资源、决策疲劳是真实现象），不因技术进步而改变。在 AI 时代，制约产出的主要因素不再是编码速度，而是工程师审查、判断和决策的能力上限。**最优策略因此不是最大化 AI 的产出量，而是最优化工程师认知带宽的分配。**

3.2 识别并质疑假设

有了上述三条公理作为判断依据，我们现在来审视业界在 AI 辅助开发中常见的几个隐含假设。这些假设并非完全错误——它们大多来自一线实践的合理总结——但如果不加条件地接受，可能会误导方

法论的设计。

假设 1: "只要把代码喂给 AI, 它就能理解整个项目"

- **来源:** 一个朴素但流行的直觉——既然 AI 的能力边界由上下文定义 (公理 2), 那只要把代码库丢进去, AI 自然就能理解项目、产出高质量代码。
- **审视:** 这个假设混淆了**信息量与知识质量**。从公理 2 的三个特征逐一分析:
 - **工作记忆有限性:** LLM 的上下文窗口是有限的。大型项目的代码库规模远超这个窗口, 不加筛选地塞入代码只会导致**信噪比急剧下降**——关键信息被海量的无关代码淹没, 输出质量反而下降。更长的上下文还意味着更高的推理延迟和成本。
 - **上下文决定性:** 代码本身只是项目知识的冰山一角。架构决策的原因、模块间的隐含契约、编码规范的微妙之处、历史 Bug 的排查经验——这些**结构化的项目知识**才是 AI 产出高质量代码的关键上下文, 而它们通常不存在于代码中。
 - **概率性:** 即使给了足够的上下文, AI 的输出仍然是概率性的。上下文质量决定的是**输出的期望质量上限**, 而非保证正确性。
- **修正后的立场:** 上下文的价值取决于**信噪比和知识结构化程度**, 而非代码的绝对量。最优策略不是"把代码喂给 AI", 而是提供**高度相关、结构化的上下文**——包括但不限于代码: 设计文档、架构约束、编码规范、模块接口契约等。这是 Context Engineering 的核心命题。

假设 2: "AI 不适合复杂系统 (如数据库) 的开发"

- **来源:** 合理的一线经验——在大型 C++ 项目中, AI 生成的代码经常不理解内部约束 (锁机制、内存管理、并发模型), 产出表面能编译但语义有缺陷的代码。系统复杂度越高, 错误越隐蔽, 开发者对 AI 的信任度越低。
- **审视:** 这个假设把问题归咎于 **AI 的能力不足**。但如果我们用公理逐一分析, 会发现问题的根源是多维度的, 且每一个都有对应的工程化解法:
 - **关键知识缺失** (公理 2·上下文决定性): 正如假设 1 的分析所揭示的, 复杂系统的关键知识通常**没有被结构化地提供给 AI**。问题不是"AI 不够聪明", 而是"AI 不知道"。
 - **上下文过载** (公理 2·工作记忆有限性): 大型项目的代码库规模远超 AI 的上下文窗口。缺少合理的 context engineering 管理机制时, AI 面对的不是"信息不足"而是"信息淹没" (假设 1 中已详细分析)。关键知识缺失和上下文过载是**同一维度的两个极端**: 一个是给得太少, 一个是给得太多且未经筛选。
 - **错误的累积效应** (公理 2·概率性 + 公理 1·意图转化链): 复杂系统的意图转化链更长、每一步的约束更多, AI 在每个环节犯错的概率都更高。如果连续让 AI 独立完成多步任务而不加校验, 错误概率将**指数级累积**——每一步的小偏差在后续步骤中被放大, 最终产出可能完全偏离意图。这意味着复杂系统更加需要**细粒度的任务拆分和每一步的 human-in-the-loop 校验**。
- **修正后的立场:** AI 在复杂系统中的可靠性, 不取决于系统的绝对复杂度, 而取决于**关键上下文是否被有效传递**。事实上, 越是复杂的项目, 工程师的认知负担越重 (公理 3), AI 分担认知负担的潜在边际价值也越大。但这一价值能否兑现, 取决于两个前提: 其一, 团队私有知识被结构化为 AI 可消费的形式 (实践 1); 其二, 通过细粒度的任务拆分和中间校验 (实践 4) 将 AI 的

错误率控制在合理范围内——否则，复杂系统中 AI 犯错的修复代价可能抵消其分担认知负担的收益。

假设 3: "AI 提效的核心价值是帮人更快地写代码"

- **来源：**最常见的 AI 提效叙事——AI 是“编码加速器”，其价值体现在代码生成速度的提升。这也是大多数 AI 编码工具（Copilot、Cursor 等）的主要卖点。
- **审视：**这个假设将 AI 的价值窄化到了意图转化链（公理 1）的**单一环节**——“设计 → 代码”。但从公理 1 看，信息损耗发生在链的**每一个环节**，而且越早期的损耗（需求误解、设计偏差）造成的返工成本越高。如果 AI 只参与编码阶段，它就只能在链的末端发挥作用——此时上游的损耗已经累积，AI 生成的代码再快也可能是在“高效地做错误的事”。更重要的是，从上下文的角度看（公理 2），如果 AI 没有参与需求澄清和设计阶段，它在编码阶段就缺少了**理解意图的关键上下文**——为什么要这样设计、有哪些约束和权衡、哪些方案被否决了以及原因。这些上下文恰恰是高质量代码生成的前提。
- **修正后的立场：**AI 的最大价值不是加速编码，而是贯穿 SDLC 全链条来降低信息损耗。让 AI 从需求澄清阶段就开始参与，每个阶段产生的结构化文档（spec、设计文档、测试用例）自然成为下一阶段的高质量上下文，形成正向循环。

假设 4: "AI 生成的代码只需要通过测试就可以提交"

- **来源：**TDD（Test-Driven Development）在 AI 编码中的流行推广——先写测试，再让 AI 生成实现代码，测试通过即可提交。这个假设的吸引力在于它提供了一个**看似客观、可自动化的验证标准**：绿灯亮了就是对的。
- **审视：**测试确实是验证 AI 生成代码的重要手段——它将验证前置，用自动化方式检验功能正确性，直接回应了 AI 概率性输出的不确定性（公理 2）。但“通过测试”只覆盖了验证的一个**维度**。从公理 1（意图转化链）看，代码的正确性不仅仅是“功能是否按预期运行”，还包括：
 - **设计意图的一致性：**代码是否忠实地实现了设计方案？是否引入了与架构约束冲突的实现方式？测试无法捕获“功能正确但设计错误”的问题。
 - **代码质量与规范：**命名是否符合项目约定？并发模型是否正确？内存管理是否安全？这些是复杂系统中隐蔽缺陷的主要来源，测试很难覆盖。
 - **性能与安全属性：**是否引入了性能回退？是否存在安全漏洞？这些非功能性约束往往需要专门的分析手段，而非普通的单元测试。
 - 此外，在大型系统（尤其是分布式系统）中，编写高质量测试本身也有很高的成本——Mock 复杂依赖、构造测试数据、模拟分布式场景，这些工作的难度和代码编写本身不相上下。
- **修正后的立场：**测试是验证工具箱中的重要一员，但远非充分条件。完整的验证策略需要多层次的手段——Spec Review（验证设计意图）、Code Review（验证代码质量和规范）、自动化测试（验证功能正确性）、集成测试（验证系统行为）——最优组合取决于任务类型和约束条件。

假设 5: "AI 应该像人一样独立完成整个任务"

- **来源：**业界对"全自主 AI 工程师"的追求——如 Devin、SWE-Agent、ralph-loop 等项目尝试让 AI 独立完成从需求理解到代码提交的完整任务。其愿景是：给 AI 一个 Issue，它自主规划、编码、测试、提交，人类只需最终验收。
- **审视：**这个假设的核心问题可以从三条公理逐一推导：
 - **错误累积不可控**（公理 2·概率性 + 公理 1·意图转化链）：AI 在每个环节的输出都是概率性的。在意图转化链上，每一步的未校验偏差会成为下一步的输入前提——偏差不会自然消解，而是趋向累积和放大。尤其当错误之间存在相关性时（如同一类知识缺失导致系列性偏差），累积效应比独立假设下更为严重。
 - **上下文在过程中退化**（公理 2·工作记忆有限性）：AI 独立完成长任务时，上下文窗口中会不断累积中间产物（代码、日志、错误信息）。随着任务推进，早期的关键上下文（需求约束、设计决策）逐渐被挤出或稀释，信噪比持续下降。AI 在任务后期做出的决策，实际上可能已经"遗忘"了任务前期的关键约束。
 - **验证被推迟到最后**（公理 3·认知稀缺）：当 AI 独立完成整个任务后，人类面对的是一个完整的、未经中间审查的大型变更。理解和验证这样一个变更所需的认知投入远高于逐步审查——你需要从头重建上下文，理解 AI 在每一步做了什么决策以及为什么。这实际上将验证成本从"分布式的低成本校验"变成了"集中式的高成本审计"，与公理 3（认知是稀缺资源）直接冲突。
- **修正后的立场：**在复杂系统中，AI 的最优工作模式不是独立自主，而是小任务推进、频繁校验的人机协作。全自主模式或许适用于约束少、验证简单的任务，但对于复杂系统工程，human-in-the-loop 不是效率的障碍，而是可靠性的保障。

3.3 最佳实践推导

有了公理作为约束、假设审视作为参照，我们现在可以推导：要达成第 2 章定义的目标——在复杂约束下可靠地提升软件工程质量并拓展工程师的能力边界——我们必须遵循哪些实践？

推导方法：每条实践都从目标出发，在公理约束下通过因果推理得出。3.2 的假设审视作为验证——每条修正立场都应被某条实践所覆盖。

以下六条实践形成递进结构：**实践 1** 构建上下文的产出、存放和加载机制；**实践 2** 基于知识不对称定义人机分工；**实践 3** 将 AI 的介入从编码扩展到全链条；**实践 4** 通过小任务和多层次验证控制错误累积；**实践 5** 用工程化方式治理团队共有知识，为前四条实践提供高质量上下文的来源；**实践 6** 从实践中的错误提取经验，形成自下而上的反馈闭环，使 AI 的上下文质量持续增强。

实践 1：Context Engineering——构建高信噪比的上下文供给系统

上下文决定性告诉我们，AI 输出的质量上限由上下文质量决定；工作记忆的有限性同时约束了上下文窗口的容量。这构成了一个核心矛盾：**复杂系统需要更多的上下文来保证质量，但 AI 能消费的上下文量是有限的。**

Context Engineering 正是解决这一矛盾的系统性方法。Anthropic 将其定义为：**策划和维护 LLM 推理过程中最优 Token 集合的策略**——涵盖系统指令、工具、外部数据、消息历史等所有可能进入上下文窗口的信息。其指导原则是：**寻找能够最大化产生期望结果可能性的最小、高信号 Token 集**。[5] 与 Prompt Engineering 仅关注单次指令的编写不同，Context Engineering 关注的是多轮推理和长时间跨度中整个上下文状态的动态管理。具体而言，它需要解决三个问题：

1. **结构化与持久化**：真正影响 AI 输出质量的往往不是代码本身，而是代码背后的知识——架构决策的原因、模块间的隐含契约、编码规范的微妙之处。这些知识之所以不存在于代码中，是因为代码表达的是“做什么”，而非“为什么这样做”。它们必须被显式地结构化为 AI 可消费的形式。
2. **对抗上下文腐化 (Context Rot)**：随着上下文窗口中 Token 数量的增加，模型准确回忆信息的能力逐步下降——这不是突然的断崖，而是性能梯度的持续衰减。因此，关键知识必须持久化为独立文档（如结构化笔记、spec），在需要时重新注入，而非依赖它在上下文中的存续。
3. **按需注入而非全量灌入**：从预先加载所有数据，转向“即时”上下文策略——Agent 在运行时根据需求动态加载信息。具体的注入载体包括：spec 文档（持久化意图和约束，按需引用）、Skills（按需加载领域知识和标准化工作流）、MCP 工具集成（动态获取外部系统信息）等。而 Rules（编码规范和架构约束）则属于全量预加载的静态上下文——它们在会话开始时即注入系统提示，确保 AI 始终遵守基本规范。

模型能力是给定常量，上下文管理才是团队真正能控制的提效杠杆。以下三条最佳实践解决上下文的**产出、存放和加载机制**（至于上下文的来源——团队共有知识如何被系统性地沉淀和管理——将在实践 5 中展开）：

实践 1.1: Spec-First——用结构化文档锚定意图

在编码之前先与 AI 协作产出 spec 文档，将意图、约束和验收标准显式化。Spec 将原本散落在对话中的模糊意图凝练为结构化的持久文档，无论对话如何膨胀，AI 都可以重新加载 spec 来恢复对任务的完整理解——这使其同时成为对抗上下文腐化的锚点。更重要的是，Spec 将任务从“AI 不知道该做什么”推向“根据明确规格执行”——通过显式注入私有知识，缩小 AI 的知识盲区。

实践 1.2: Docs as Code——让文档成为上下文的一等公民

Spec-First 解决了“产出什么”的问题，Docs as Code 解决的是“存在哪里、如何被发现”。将完成的 spec 设计文档作为代码保存在仓库中，与源代码共同版本化，带来三方面的价值：

- **可检索性**：当 AI 需要某个模块的上下文时，可以直接从仓库中检索对应的文档，而非依赖工程师临时口述或在对话中反复解释。文档离代码越近，AI 获取高质量上下文的路径就越短。
- **一致性**：文档与代码同仓库、同分支、同 MR，天然保持同步。过期的文档比没有文档更危险——它会注入错误的上下文。
- **可协作性**：团队成员产出的文档自然成为其他成员（和 AI）的可复用上下文，知识不再锁在个人对话历史中。

实践 1.3：渐进式披露（Progressive Disclosure）——按需展开，而非一次性灌入

按需注入的核心挑战是"何时注入多少"。以 Skills 为例，一个包含完整领域知识的 Skill 可能有数千 Token，如果在任务开始时就全量加载，会立即消耗大量注意力预算。更好的做法是渐进式披露——Skill 仅在被触发时才加载，且内部先展开最小必要指令，随着任务推进按需展开细节。这使得 Skills 成为 Context Engineering 中最能体现"即时上下文策略"的载体。

验证：覆盖假设 1、假设 2 的修正立场。

实践 2：基于知识不对称的人机分工

实践 1 构建了上下文供给的基础设施，但还没有回答"人和 AI 各自该做什么"。要找到最优的分工策略，前提是理解双方各自的能力边界。

公理 2（上下文决定性）定义了 AI 的知识边界：AI 具备广博的通用知识（来自训练数据），但对团队私有知识（架构约束、业务逻辑、内部规范）一无所知。公理 3 定义了人类的能力边界：工程师的认知资源是有限的——这意味着没有人能在所有领域都具备专家级知识，必然存在"人类不知道"的领域。

这两个边界的交叉产生了四种不同的知识状态，每种状态对应不同的最优协作策略——这就是乔哈里窗（Johari Window）模型在人机协作中的应用[6]：

	AI 知道 (通用知识)	AI 不知道 (私有知识)
人类知道	开放区 ：极致自动化	盲区 ：上下文注入
人类不知道	潜能区 ：知识杠杆	未知区 ：协同探索

开放区（极致自动化）：意图明确且属通用知识——AI 拥有全部信息，人类介入只增加延迟。策略：直接下达指令，极致自动化（如样板代码生成、单元测试编写、格式化重构）。

盲区（上下文注入）：任务涉及团队私有知识，AI 无从得知。不显式注入，AI 只能基于通用知识猜测——这是"AI 生成的代码不符合项目规范"的根源。注入手段即实践 1 中的各类载体（spec、Rules、Skills、MCP 等）。

潜能区（知识杠杆）：人类在某领域缺乏专业知识，但 AI 训练数据恰好覆盖——利用 AI 的广博知识补全人类短板，拓展能力边界（如接手陌生技术栈、排查不熟悉的编译器报错）。

未知区（协同探索）：双方都缺乏确定性知识，没有任何一方能独立给出答案。在人机协作的二元框架内，迭代探索是唯一可行的收敛策略——人类提供假设和方向判断，AI 提供验证思路和方案生成，双方在循环中逐步逼近答案（如复杂根因分析、全新架构设计）。

分工的核心原则是：识别当前任务落在哪个象限，采用对应的协作策略。一个简单的启发式：问自己两个问题——(1) 这个任务是否涉及团队私有知识？(2) 我自己是否清楚该怎么做？两个答案的组合直接映射到四个象限。盲目地全交给 AI（忽略盲区）或全自己做（浪费潜能区），都不是最优解。

验证：覆盖假设 2、假设 5 的修正立场。

实践 3：AI 全链条参与——从引导者到执行者

实践 1 说明了为什么知识必须结构化持久化，实践 2 给出了通用的分工原则。但还没有回答一个关键问题：**AI 应该在 SDLC 的哪些阶段介入，分别扮演什么角色？**

公理 1（意图转化链）告诉我们，源头损耗传播最远、修复代价最高——需求和设计阶段的质量决定了后续所有环节的上限。人类当然可以独立完成这些阶段，但公理 3（认知稀缺）揭示了现实约束：需求澄清和方案设计耗时长，不同工程师考虑问题的周全度参差不齐，且缺乏统一的结构化方法来确保完备性。当 AI 被提供了足够的领域上下文（实践 1、实践 5）时，它参与早期阶段的边际成本较低（主要是对话交互），但由于源头损耗的传播效应，收益在下游被持续放大。需要注意的是，这一结论是有条件的：如果 AI 缺乏领域知识，过早介入反而可能引入 1.2 节所述的“似是而非”损耗——因此上下文供给（实践 1）是 AI 全链条参与的前提，而非结果。在此前提下，**AI 应从意图澄清阶段就开始参与，而非仅仅负责代码生成。**

但 AI 在不同阶段的角色不应相同。将实践 2 的乔哈里窗模型沿意图转化链展开，可以看到知识状态随阶段变化，AI 的角色也应随之变化：

- **需求澄清阶段：**任务主要落在**盲区**（人类有业务意图但未结构化，AI 不了解私有知识）和**潜能区**（AI 的通用知识可以帮助发现遗漏）。AI 的最优角色是**引导者**——通过结构化的提问，帮助工程师将模糊意图显式化为 spec，并利用通用知识和注入的领域最佳实践发现人类未曾考虑到的边界条件、异常路径和隐含约束。
- **方案设计阶段：**spec 将私有知识显式注入后，任务从盲区移向**开放区**。AI 的角色升级为**协作者**——主动分析方案权衡、提出替代设计、检查与架构约束的一致性、识别可扩展性或性能隐患，人类做最终的设计决策。
- **编码与测试阶段：**有了 spec 和设计文档作为高质量上下文，任务大部分进入**开放区**。AI 的角色转变为**执行者**，可以承担更大的自主性。

一个值得展开的问题是：为什么需求阶段 AI 是引导者，而非直接给出方案？因为人类是后续所有决策的最终审批者——如果他/她自己没有深度理解问题，后续的审查和决策都会变成盲审。AI 直接给方案会让人类跳过思考过程，后续需要从头重建理解，认知成本反而更高。让人类在 AI 引导下自己得出结论，才是认知成本最低的路径。

这一角色递进也进一步印证了实践 1.1 Spec-First 的必要性——spec 是将任务从盲区推向开放区的关键机制，没有它，AI 在编码阶段仍然只能猜测。

此外，AI 全链条参与还产生一个正向循环：每个阶段产生的结构化文档（spec、设计文档、测试用例）自然成为下一阶段的高质量上下文——需求澄清产出 spec，spec 指导设计，设计文档指导编码。这使得实践 1（Context Engineering）的要求被逐步满足，而非一次性解决。

验证：覆盖假设 3 的修正立场。

实践 4：小任务推进、多层次验证——控制错误累积的两个维度

目标要求“可靠”和“提升质量”。公理 2 的两个子特征共同约束了 AI 的长链执行能力：概率性意味着每一步的未校验偏差都会成为下一步的输入前提——偏差不会自然消解，而是趋向累积和放大，尤其当错误之间存在相关性（如同一类知识缺失引发系列性偏差）时，累积效应更为严重；工作记忆的有限性则带来双重约束——一方面，复杂任务的完整上下文（需求、架构约束、相关代码）可能根本无法一次性装入上下文窗口；另一方面，即便装得下，任务越长，早期的关键约束也越可能被稀释或挤出。

即便想在最后一次性补救也不行——公理 3 从人类端封堵了这条退路：面对大型的、未经中间审查的变更，审查者需要从头重建上下文，认知成本远高于分步审查。

两端约束共同指向一个结论：**必须将大任务拆解为可独立验证的子任务，每步由 AI 执行、人类校验，通过后再进入下一步。**这同时控制了三个风险：错误累积（每步校验防止偏差放大）、上下文退化（通过为每个子任务构建聚焦的上下文，而非在一个膨胀的会话中持续追加）、验证成本（小变更比大变更更容易审查）。

任务粒度不是固定的——约束越密集的任务（如涉及并发、内存管理的底层代码），步长应越短；约束较少的任务（如生成样板代码），可以给予 AI 更大的自主空间。

解决了“多频繁验证”之后，还需要回答“怎么验证”。公理 1（意图转化链）告诉我们，意图在链上经过多次变换——从需求到设计到实现到行为——每一层的“正确”含义不同。编写测试验证的是行为层（“功能是否按预期运行”），但它无法捕获：

- **设计层的偏差：**代码功能正确，但实现方式与架构约束冲突（例如在应该用异步的地方用了同步阻塞）
- **实现层的缺陷：**命名不符合项目约定、并发模型使用不当、内存管理不安全——这些是复杂系统中隐蔽缺陷的主要来源
- **非功能性约束：**性能回退、安全漏洞——需要专门的分析手段，普通单元测试无法覆盖

因此，验证策略必须与意图转化链的层次对齐，在每一层使用对应的验证手段：

验证层次	验证对象	验证手段
意图层	需求完备性、约束覆盖	需求评审、设计文档 Review
实现层	代码质量、规范合规、架构一致性	Code Review
行为层	功能正确性	自动化测试
系统层	非功能性约束（性能、安全、集成）	集成测试、性能测试

具体任务需要哪些层次的验证，取决于其约束空间——约束越复杂的任务，需要的验证层次越多。

验证：覆盖假设 4、假设 5 的修正立场。

实践 5：Knowledge as Code——用工程化方式治理团队共有知识

实践 1 解决了"AI 需要什么样的上下文、如何注入"的机制问题。但还有一个更根本的问题没有被回答：这些高质量上下文从哪里来？

公理 3（认知稀缺）告诉我们，工程师的认知资源是有限的——没有人能在所有领域都具备专家级知识。这意味着团队成员的专长分布必然不均，最佳实践、设计原则、编码规范——这些团队最有价值的共有知识，往往锁在少数资深工程师的脑中。在传统模式下，知识传递依赖 Code Review 中的言传身教、新人 onboarding 时的口头讲解，效率低且不可规模化。这就是**知识孤岛**：知识存在，但无法被系统性地获取和复用。

AI 的介入创造了一个打破知识孤岛的新范式：**将团队共有知识像代码一样管理**——编码为结构化的 Skill 和 Rules，存入仓库，版本化、可 Review、可迭代。这不是简单的"写文档"，而是一种知识治理的范式转换：

- **知识的公有化：**当资深工程师的设计原则和编码规范被编码为 Skill 后，AI 成为知识分发的载体——每个团队成员在与 AI 协作时，都自动获得了团队最佳实践的加持。这很大程度上拉平了团队内部的能力差异。

- **知识的可演进性**：Skill 和 Rules 与代码同仓库、同流程——通过 MR 提出修改，通过 Review 达成共识，通过版本历史追溯演进。团队的工程经验不再是静态的文档，而是持续生长的活资产。

验证：覆盖“知识可沉淀”和“可持续演进”目标维度，以及假设 2 的修正立场。

实践 6：Error-Driven Context Refinement——从错误中构建反馈闭环

实践 5 是**自上而下**的知识供给——将团队已有的最佳实践编码为上下文。但很多关键知识在项目开始时并不存在，它们是在**开发过程中通过犯错和纠错逐步浮现**的。

公理 2（工作记忆易失性）指出：LLM 没有跨会话的持久记忆——会话 A 中被纠正的错误，在会话 B 中会以相同概率再次发生。结合上下文决定性——输出由上下文决定——可以推出：**唯一的解法是将错误经验外化为持久化的上下文**。公理 1（信息损耗）进一步揭示，意图转化链上的损耗很多是**模式化的、可重复的**，因此可以被系统性地预防。公理 3（认知稀缺）给出最后的推力：工程师反复纠正 AI 的同类错误是认知资源的浪费。

因此，**必须建立从错误到持久化规则的反馈机制**。这个闭环通过两种互补的途径触发：

途径一：自动触发。通过一个专门的自迭代 Skill 实现——当 AI 在协作过程中识别到自己被纠正（用户否定了 AI 的输出并给出了正确方向），它自动评估这个错误的根因，搜索现有的 Rules 和 Skills 中是否已有针对该类问题的处理规则。如果没有，则向用户建议创建或更新对应的 Rule/Skill，经用户确认后执行。这使得知识积累成为开发流程的自然副产物，而非额外的负担。

途径二：手动触发。通过一个显式的 Command 实现——用户在任意时刻主动发起，AI 回顾当前对话历史，识别其中被纠正的错误模式，搜索对应的 Skills 和 Rules 评估是否需要更新，并向用户提出具体的修改建议。这适用于工程师在一轮密集协作后，集中性地将积累的经验沉淀为持久化知识。

两种途径的共同本质是同一个闭环：**犯错 → 诊断根因 → 检索现有知识 → 创建或更新 Rule/Skill → 预防复发**。它与实践 5 形成互补——实践 5 自上而下编码已有知识，实践 6 自下而上从错误中生长新知识——共同构成完整的知识生命周期。

验证：覆盖“可持续演进”目标维度，以及假设 1、假设 2 的修正立场。

04

基于 Skill 的 Agentic Engineering 框架

第 3 章推导了六条最佳实践，本章回答：如何将它们落地为一个可执行的工程框架？

核心思路是将最佳实践映射到一种模块化、按需加载、可组合的工程载体——Agent Skill。每个 Skill 是一个独立目录，包含结构化的指令（Markdown + YAML 元数据）和可选的参考资源，AI Agent 在相关任务触发时自动加载。Skill 本质上是纯文本文件，主流 Coding Agent（Claude Code、Cursor、CodeBuddy 等）均支持，不会被锁定在特定工具上。

4.1 为什么是 Skill

Skill 不是随意选择的载体——它是最佳实践的约束条件自然推出的工程化形式：

最佳实践的约束	对载体的要求	Skill 如何满足
上下文窗口有限	按需加载，避免全量灌入	三层渐进式披露（见下表）
任务需要分步验证	内嵌检查点和护栏	Workflow Skill 在每个阶段设置前置检查，强制流程合规
知识需要结构化管理	版本化、可 Review、可迭代	Skill 与代码同仓库、同 MR 流程，知识演进有据可查
知识需要持续生长	低成本创建和更新	添加新 Skill 只需在对应目录下创建 SKILL.md，无需额外配置

大量 Skills 不会拖垮上下文窗口，因为框架采用三层加载机制：

层级	内容	加载时机	Token 成本
L1: Metadata	YAML frontmatter (name, description)	Agent 启动时	~100/Skill
L2: Instructions	SKILL.md 主体指令	Skill 被触发时	< 5k
L3: Resources	references/（参考文档）、scripts/（可执行脚本）、assets/（静态资源）	被显式引用或调用时	按需

项目可以安装数十个 Skills（仅消耗 L1 元数据），只有当前任务实际使用的 Skill 才消耗完整上下文。

Skill 与 Rules 互补：**Rules** 是海量预加载的静态上下文，只适合放置极轻量、在任何阶段都需要的全局信息（如项目目录结构、通用沟通约定），必须严格控制体量；**Skills** 是按需触发的专项能力，编码规范、架构约束等领域知识应放在 **Skills** 中，仅在相关阶段被加载——避免在不需要它们的阶段浪费上下文窗口。

4.2 框架架构

框架由六个模块组成：

模块	职责	触发方式
Workflow	SDLC 全链条工作流（需求→设计→编码→测试→评审）	直接触发
Best Practices	通用工程知识（架构、编码、性能、分布式等）	被 Workflow 按需调用
Standards	项目/团队私有知识（编码规范、模块约束、测试规范）	被 Workflow 按需调用
Docs	项目文档（spec.md、设计文档、架构决策记录等），Docs as Code	被 Workflow 按需读取
Troubleshooting	问题排查（编译错误、运行异常、现网告警等）	直接触发
Self-Refinement	从错误中提取经验，更新 Rules/Skills	自动触发 + 手动 Command

模块间的关系：Workflow 是主驱动链，在执行每个阶段时按需拉取 Best Practices、Standards 和 Docs 作为上下文；Troubleshooting 独立于 Workflow，在问题排查场景下直接触发；Self-Refinement 横跨所有模块——它监听协作过程中的错误信号，将提取的经验沉淀到 Best Practices、Standards 或 Rules 中，形成持续改进的闭环。

4.3 Workflow 如何平衡工程纪律与灵活性

Workflow Skills 是框架的核心——但工程纪律不等于一刀切的重流程。框架的关键设计是：**先评估任务复杂度，再决定流程深度。**

以 code-generation Skill 为例，它在执行前会先对任务进行复杂度评估：

code-generation Skill 的执行逻辑（简化示意）：

0. 评估任务复杂度

- └─ 简单任务（单文件、局部修改、意图明确）
 - | → 跳过 spec 检查，直接加载 Standards 后编码
- └─ 复杂任务（多文件、跨模块、涉及设计决策）
 - 进入完整流程 ↓

1. spec.md 是否存在？

- └─ 否 → 停止编码，切换到 requirements-clarification Skill
- └─ 是 → 读取并确认理解

2. spec.md 是否包含必要章节（目标、需求、设计）？

- └─ 缺失 → 引导用户补充
- └─ 完整 → 继续

3. 加载项目 Standards（编码规范、模块约束）

4. 开始编码（分步执行，每步可审查）

这种设计在**安全性**和**敏捷性**之间取得平衡：对于简单的单文件修改（如修一个 Bug、加一个字段），工程师无需走完整的 spec 流程，框架自动轻量化处理；而对于涉及多文件、跨模块的复杂变更，即使用户想“跳过需求直接写代码”，Agent 也会引导其回到规范流程——Spec-First 被强制执行，而非仅仅被建议。**流程的严格程度与任务的风险成正比。**

4.4 项目私有知识的扩展

Best Practices 和 Standards 是框架中**最需要按项目定制**的部分。框架提供通用的目录结构和 Skill 编写规范，团队根据自身情况按需填充：

skills/

- └─ best-practices/ # 通用最佳实践（可跨项目复用）
- | └─ architecture-design/ # 架构设计原则
- | └─ coding-practices/ # 编码实践
- | └─ performance/ # 性能优化
- | └─ ... # 按需扩展
- |

```
|—— standards/          # 项目私有知识（团队定制）
|
| |—— language-cpp/      # C++ 编码规范
|
| |—— module-storage/    # 存储模块约束
|
| |—— testing-conventions/ # 测试规范
|
| |—— ...                # 按需扩展
|
|
|—— ...
```

扩展原则：添加新的 Best Practice 或 Standard 只需在对应目录下创建 SKILL.md 文件。框架不规定具体内容——它规定的是知识应该以什么形式存在、在什么时机被加载、如何被演进。具体填充什么知识，取决于每个团队的技术栈和工程需求。

4.5 开源项目链接

本章描述的是框架的**架构和设计原则**，而非具体实现。我们已将上述方法论落地为一个开源框架，欢迎大家 fork/contribute：

<https://github.com/davidYichengWei/agentic-engineering-framework>— 基于 Skill 的模块化 Agentic Engineering 框架，包含完整的 SDLC Workflow、Best Practices、Self-Refinement 机制及项目定制指南。

框架的核心设计优势：

- **Agent-Agnostic：**框架的载体是纯 Markdown 文件（YAML 元数据 + 结构化指令），不依赖任何特定 Agent 平台的 API 或插件机制。Claude Code、Cursor、CodeBuddy、Codex CLI 等主流 Coding Agent 均原生支持，迁移成本为零。
- **开箱即用：**内置完整的 SDLC Workflow（需求澄清→系统设计→代码生成→测试生成→代码评审）、通用 Best Practices（架构设计、编码实践、性能优化、分布式系统等）、以及 Self-Refinement 反馈闭环，clone 后即可使用。
- **易于扩展：**添加新的编码规范或最佳实践，只需在 skills/ 目录下创建一个 SKILL.md 文件，并在对应的 Workflow Skill 中注册触发条件——无需修改框架代码或配置文件。
- **按需加载：**三层渐进式披露机制（L1 元数据→L2 指令→L3 参考资源）确保数十个 Skills 不会拖垮上下文窗口，只有当前任务实际触发的 Skill 才消耗完整上下文。
- **知识可演进：**Skills 与代码同仓库、同 MR 流程，团队的工程经验通过版本化管理持续生长，而非停留在一次性的文档编写。

05

写在最后：找到自己的位置，然后行动

AI 发展得太快了。每隔几个月就有新的模型刷榜，每隔几周就有新的范式概念出炉——Agentic Engineering、Harness Engineering……让人应接不暇，也让人不安。很多软件工程师会隐隐觉得：是不是快要被替代了？

这种焦虑完全正常。但越是在快速变化的时期，越不能被变化本身裹着走。真正有用的思考方式，是把“变的”和“不变的”分开看——不要把精力押在会变的东西上，而是押在不变的那一边。

变的是什么？

AI 模型的能力边界在快速移动。今天的上下文窗口、幻觉率、工具调用能力、自主决策水平——这些都是阶段性的技术瓶颈，不是物理定律。它们终将被突破，区别只在于快慢。所以本文中那些基于当前 AI 局限推导出的具体做法——任务拆多细、护栏加多密、上下文怎么管——都不是一成不变的。工具变强了，做法就该跟着调。

这也意味着，不应该把有限的精力过分花在那些会被模型进步自然抹平的技能上。比如，与其花大量时间钻研 prompt 的措辞技巧，不如去理解 Agent 的编排和系统设计——前者会随着模型变聪明而贬值，后者不会。与其纠结于某个特定工具的用法细节，不如去建立对 AI 协作范式本身的理解——工具会换代，思维方式不会。

不变的是什么？

两件事不会变。

第一，**软件工程的本质没变**——它始终是通过交付软件来解决有价值的实际问题。不管 AI 写了多少代码，最终要回答的问题还是：这个东西解决了谁的什么问题？解决得好不好？这个判断不会因为执行变快了就不需要了，反而会因为执行太快而变得更关键——方向错了，浪费也更大了。

第二，**人类的判断力和认知仍然是稀缺的**。机器可以不断加算力，人不行。一个团队永远需要有人去判断轻重缓急、做取舍、把模糊的想法变成清晰的目标。这是生物学层面的约束，不是技术能解决的。

软件工程师会往哪里走？

想清楚了变与不变，方向就比较自然了。软件工程师未来大致会往两个方向发展：

一是成为最会编排 AI 的人。 大概率不再亲手写代码，但要能把需求高效、高质量地落地为可运行的系统。这需要对流程和效率的极致追求——怎么最大化 Agent 的并行度，怎么提升 Agent 产出的质量，怎么设计验证机制确保结果可靠。同时也需要保持持续学习，跟上 AI 时代不断涌现的新范式和新工具。软件工程师多年积累的系统思维和工程素养，使得这个群体天然适合这个方向。

二是往软件工程的上游发展。 结合自己的专业领域知识，成为最会识别“什么是最有价值的问题”的人。工程师比 PM 多一个独特优势——知道系统能做什么、不能做什么、做某件事的真实成本是多少。这种技术 insight 是发现高价值需求的稀缺原材料。

这两条路不是非此即彼，大多数人最终会在两个方向上同时发展，只是侧重不同。但不管偏哪边，底层都是同一件事：对系统的理解、对问题的判断、对价值的感知。

现在就可以做的几件事

说了这么多，落到行动上，其实不用太复杂：

跑通一个真实闭环。 找一个边界清晰的小需求——修一个 Bug、加一个配置项、做一个小功能——用 Agentic 的方式完整走一遍：先写清楚需求，再让 AI 执行，再审查结果。忍住自己下场写代码的冲动。真正难的不是让 AI 写代码，而是学会放权和把关。

把省下来的时间花在更高维度的事情上。 不要和 AI 拼写代码的手速——这是一场注定会输的竞赛。把释放出来的精力投入到定义和拆解问题、跨领域的系统级思考、研究新技术和新范式、优化团队的研发流程和协作方式上。这些才是不会贬值的能力。

AI 跑偏的时候，别急着接管，先想想哪里没做到位。 问题讲清楚了吗？上下文给够了吗？工具打通了吗？很多时候“AI 不行”其实是“前置条件没准备好”。

把每一次错误都变成进化的机会。 与其反复提醒 AI 同样的事，不如把缺失的规范和知识沉淀下来。错误不是负担，而是系统生长的养料。

最后，保持乐观。现在仍然是 AI 技术发展的红利期——AI 的大规模普及还远未完成，而软件工程恰好是 AI 率先落地的领域。这意味着，能够熟练运用 AI 的工程师所获得的杠杆效应是巨大的，和还没有拥抱 AI 的人之间的价值差距会越拉越大。争做 AI 的 early adopter，就是在这波浪潮中为自己争取最大的复利窗口。

别想太远。能思考问题的本质、能解决真实的问题——拥有这种能力的人在任何时代都是被需要的，这也是不变的。先把“与 AI 高质量协作”这件事练熟，后面的路会自然展开。

参考文献

1. **Addy Osmani**: "Agentic Engineering", 2025. (<https://addyosmani.com/blog/agentic-engineering/>)
2. **GitHub Copilot Productivity Study**: "Quantifying GitHub Copilot's impact on developer productivity and happiness", 2022. (<https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/>)
3. **Anthropic Internal Study**: "How AI is transforming work at Anthropic", 2025. (<https://www.anthropic.com/research/how-ai-is-transforming-work-at-anthropic>)
4. **Nelson F. Liu, Kevin Lin, John Hewitt, et al.**: "Lost in the Middle: How Language Models Use Long Contexts", Stanford University & University of Washington, 2023. (<https://arxiv.org/abs/2307.03172>)
5. **Anthropic**: "Effective context engineering for AI agents", 2025. (<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>)

-End-

原创作者 | 魏依承

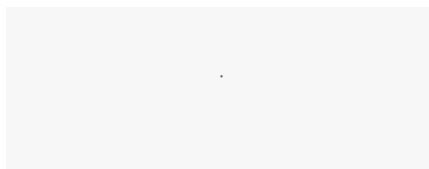
感谢你读到这里，不如关注一下？👉



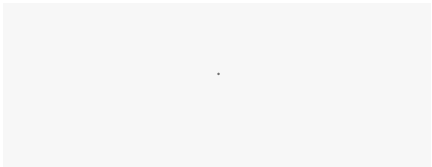
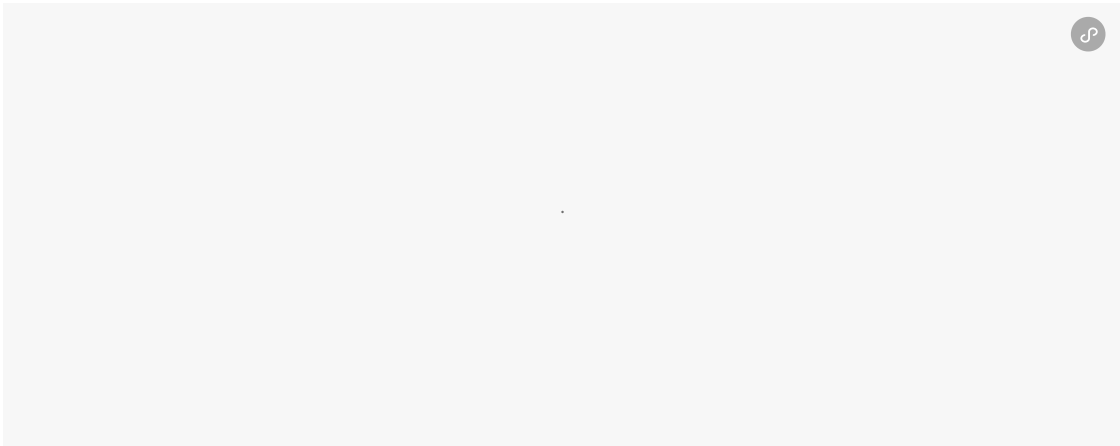
腾讯云开发者

腾讯云官方社区公众号，汇聚技术开发者群体，分享技术干货，打造技术影响力交流社... 1095篇原创内容

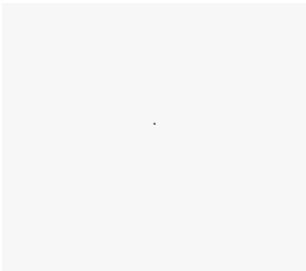
公众号



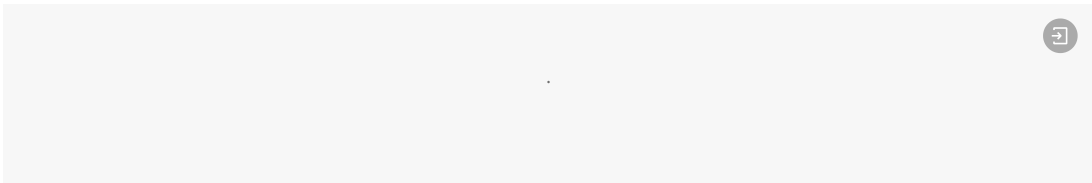
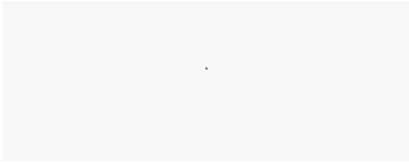
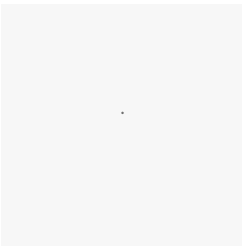
📢 来抢开发者限席名额！点击下方图片直达👉

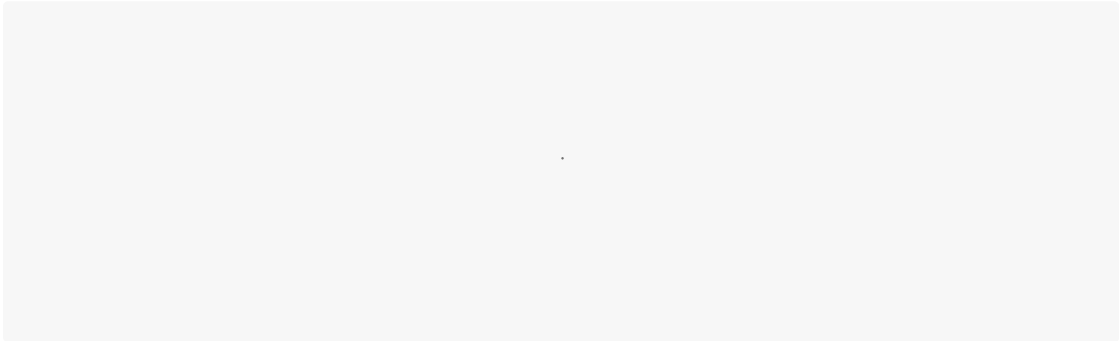
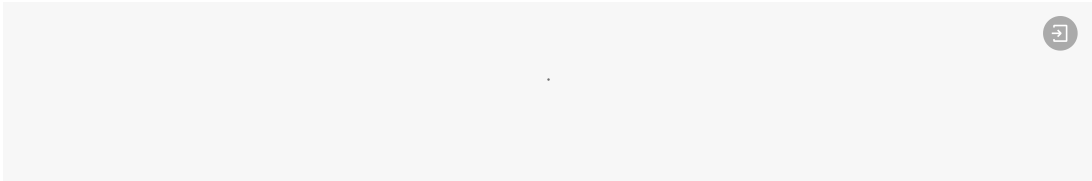
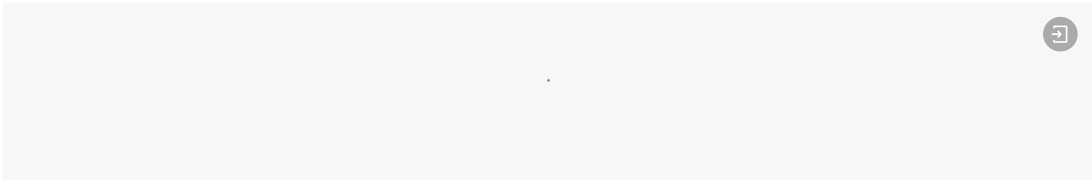


你对本文内容有哪些看法？同意、反对、困惑的地方是？欢迎留言，我们将邀请作者针对性回复你的评论，欢迎评论留言补充。我们将选取1则优质的评论，送出腾讯云定制文件袋套装1个（见下图）。4月29日中午12点开奖。



扫码领取腾讯云开发者专属服务器代金券！





腾讯技术人原创集 · 目录 ≡

< 上一篇

万字干货！Harness Engineering如何工程化落地？

下一篇 >

Harness Engineering实践心得：如何高效驾驭AI？